Python Tip: List Comprehensions

Want to build lists in a cleaner way?

List comprehension allows us to build a complicated list simply, often in a single line.

It is one of the clearest markers of “Pythonic” code.



Building a List the Long Way

Imagine we have a list of item prices and we want a second list with 8% sales tax added to each price.

The usual approach is to create an empty list, loop over the prices, and append each new value.

```
prices = [12.99, 4.50, 29.00, 7.25]

with_tax = []
for price in prices:
    with_tax.append(round(price * 1.08, 2))

print(with_tax)
```

```
[14.03, 4.86, 31.32, 7.83]
```

This works, but there is a cleaner way to do it...

The Same List in One Line

Python gives us a shorter way to make lists, called a **"list comprehension"**. A list comprehension is a single expression, written inside square brackets, that builds an entire list at once.

Here is the same calculation on the same prices.

```
with_tax = [round(price * 1.08, 2) for price in prices]

print(with_tax)
```

```
[14.03, 4.86, 31.32, 7.83]
```

Writing a list comprehension is like writing a `for` loop backwards. We start with the logic we want to run, `round(price * 1.08, 2)`, and then we say what we are looping over, `for price in prices`. The square brackets around both parts collect every result into a new list.

Keeping Only Some Items

A comprehension can also skip items. If we add `if` at the end, Python keeps only the prices for which the condition is `True`.

Let's say we want the prices over $10.

```
expensive = [price for price in prices if price > 10]

print(expensive)
```

```
[12.99, 29.0]
```

The logic on the left is just `price`, so each kept item goes into the new list unchanged. A comprehension can filter a list without changing any values.

Looping Over Two Levels

Data often arrives nested. Suppose our prices are grouped by order, so `orders` is a list of lists, and we want one flat list containing every price.

A comprehension can use two `for` clauses to do this.

```
orders = [[12.99, 4.50], [29.00], [7.25, 3.75]]

all_prices = [price for order in orders for price in order]

print(all_prices)
```

```
[12.99, 4.5, 29.0, 7.25, 3.75]
```

The two `for` clauses go in the same order we would write two nested loops: `for order in orders` on the outside, then `for price in order` on the inside. The second clause is allowed to use `order` because the first clause has already defined it.

When Not to Use a Comprehension

The convention is to only write a comprehension when we actually want the list it builds.

If we simply want to run something for each item, such as printing it or writing it to a file, a regular `for` loop is easier to read.

```
# We want the list, so a comprehension fits
with_tax = [price * 1.08 for price in prices]

# We only want to print, so a loop is clearer
for price in prices:
    print(price)
```

Wrap-Up

If you are building a new list from an existing collection, use a comprehension.

If you are doing something for each item, use a `for` loop.

Write one backwards: the logic first, then the loop that feeds it.

Follow me for more tips.

- Shep4